

1) Create a class Address for Author with instance variables streetNumber, location, State.

```
import javax.persistence.*;

@Getter no usages
@Setter
@AllArgsConstructor
@NoArgsConstructor
@Embeddable
public class AddressModel{

    private String streetNumber;
    private String location;
    private String state;

}
```

2) Create instance variable of Address class inside Author class and save it as embedded object.

```
import java.util.List;

@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@Entity
public class AuthorModel {

    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    Long id;

    private String name;

    @Embedded
    private AddressModel addressModel;

    private List<String> subjects;
}
```

3) Introduce a List of subjects for author.

```
    @Embedded
    private AddressModel addressModel;
    @ElementCollection
    private List<String> subjects;
}
```

4) Persist 3 subjects for each author.

```
@PostMapping("/addUser")
public ResponseEntity<AuthorModel> saveNewUser(@RequestBody AuthorModel model){
    AuthorModel m= service.saveAuthor(model);
    return ResponseEntity.status(HttpStatus.OK).body(m);
}
```

```
public AuthorModel saveAuthor(AuthorModel m){ no usages
    return repo.save(m);
}
```

5) Create an Entity book with an instance variable bookName.

```
@Setter
@Getter
@Entity
public class Book {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    Long id;

    private String bookName;
}
```

6) Implement One to One mapping between Author and Book.

```
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "books_id")
private Book book;
```

7) Implement One to Many Mapping between Author and Book(Unidirectional, BiDirectional and without additional table) and implement cascade save.

```
@ElementCollection
private List<String> subjects;

@OneToMany(cascade = CascadeType.ALL)
@JoinColumn(name = "books_ref_id" ,referencedColumnName = "books_id")
private List<BookOneToOne> bookOneToOne2;
}
```

Unidirectional

```
private String bookName;

//bidirectional
@ManyToOne(cascade = CascadeType.ALL )
@JoinColumn(name="author_ref", referencedColumnName = "authorId")
private AuthorOneToOne author;
}
```

BiDirectional

8) Implement Many to Many Mapping between Author and Book.

```
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@Entity
public class AuthorManyToMany {

    @Id
    @GeneratedValue
    private Long id;

    private String name;

    @ManyToMany
    @JoinTable(
        name = "author_book",
        joinColumns = @JoinColumn(name = "author_id"),
        inverseJoinColumns = @JoinColumn(name = "book_id")
    )
    private List<BookOneToOne> bookOneToOne;
}
```

```
@AllArgsConstructor
@NoArgsConstructor
@Setter
@Getter
@Entity
public class BookManyToMany {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    Long id;

    private String bookName;

    @ManyToMany(mappedBy = "books")
    private List<AuthorManyToMany> authors;
}
```

9) Which method on the session object can be used to remove an object from the cache?

10)What does @transactional annotation do?

BEGIN TRANSACTION

execute code

COMMIT / ROLLBACK

@Transactional ensures a method runs within a database transaction and automatically commits or rolls back changes.